

FlowDPS : Flow-Driven Posterior Sampling for Inverse Problems

Jeongsol Kim^{*1} Bryan Sangwoo Kim^{*2} Jong Chul Ye²

¹Department of Bio and Brain Engineering, KAIST

²Kim Jaechul Graduate School of AI, KAIST

{jeongsol, bryanswkim, jong.ye}@kaist.ac.kr

* Equal contribution

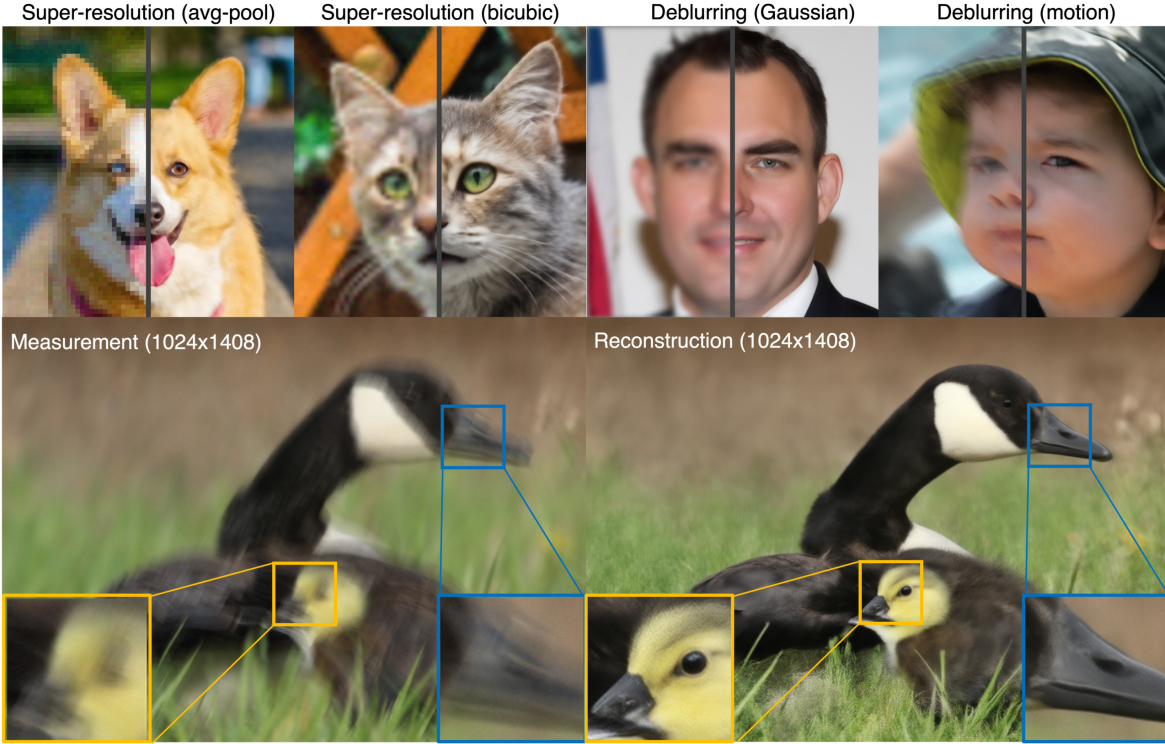


Figure 1. (Top) Super-resolution and deblurring results for linear inverse problems before and after FlowDPS. (Bottom) Results for motion deblurring on a high-resolution image.

Abstract

Flow matching is a recent state-of-the-art framework for generative modeling based on ordinary differential equations (ODEs). While closely related to diffusion models, it provides a more general perspective on generative modeling. Although inverse problem solving has been extensively explored using diffusion models, it has not been rigorously examined within the broader context of flow models. Therefore, here we extend **the diffusion inverse solvers (DIS)** — which perform posterior sampling by combining a denoising diffusion prior with **an likelihood gradient** — into **the flow framework**. Specifically, by driving the flow-version of Tweedie’s formula, we decompose the flow ODE

into two components: **one for clean image estimation** and the **other for noise estimation**. By integrating the likelihood gradient and stochastic noise into each component, respectively, we demonstrate that posterior sampling for inverse problem solving can be effectively achieved using flows. Our proposed solver, Flow-Driven Posterior Sampling (FlowDPS), can also be seamlessly integrated into a **latent flow model with a transformer architecture**. Across four linear inverse problems, we confirm that FlowDPS outperforms **state-of-the-art alternatives**, all without requiring **additional training**. Code : <https://github.com/FlowDPS-Inverse/FlowDPS>

1. Introduction

The goal of inverse problems is to recover the true signal $\mathbf{x}_0$ from measurements $\mathbf{y}$, which are generated through a forward model. This forward model typically represents a physical, computational, or statistical process that maps the true signal to the measurement space, often introducing noise, distortion, or information loss.

For example, a typical linear inverse problem is defined as finding $\mathbf{x}_0 \in \mathbb{R}^d$ from given measurement $\mathbf{y}$:

$$\mathbf{y} = \mathbf{A}\mathbf{x}_0 + \mathbf{n} \in \mathbb{R}^m, \quad (1)$$

where $\mathbf{A} \in \mathbb{R}^{m \times d}$ represents the forward model (or imaging system) and $\mathbf{n} \in \mathbb{R}^m \sim \mathcal{N}(0, \sigma_n^2 \mathbf{I}_m)$ denotes Gaussian noise with variance σ_n . Because inverse problems are inherently ill-posed, there are multiple solutions $\mathbf{x}$ that satisfy Eq. (1). The key challenge, therefore, is to constrain the solution space to achieve a unique and meaningful reconstruction.

Traditionally, this has been done by adding a regularization term in a Maximum A Posteriori (MAP) framework [1]. More recently, advancements in diffusion models have introduced techniques for guiding the sampling trajectory to perform posterior sampling [2, 3, 24, 25]. These diffusion model-based approaches aim to select the solution $\mathbf{x}_0$ from the image distribution $q(\mathbf{x}_0)$ (learned by the generative model) that is closest to the subspace defined by $\mathbf{y} - \mathbf{A}\mathbf{x}_0 = 0$. Such innovative methods have quickly become the state-of-the-art in the field, producing superior results compared to traditional approaches [2, 3, 24, 25].

Meanwhile, Latent Diffusion Models (LDMs) [17, 19] have emerged as foundational generative models trained on large-scale multi-modal datasets [21]. By virtue of the flexibility and memory efficiency of the latent space, they can generate diverse content, including images, videos, audio, and multi-modal outputs simultaneously. Building on the success of LDMs, recent zero-shot inverse problem solvers aim to utilize their superior generative prior to tackle inverse problems (e.g., PSLD [20], ReSample [22], DAPS [27]). To further constrain the solution space, P2L [4] leverages null-text embedding as a learnable parameter, and TReg [8] introduces the concept of text-regularization for latent inverse problem solvers by using textual descriptions as guidance. Building on such ideas, DreamSampler [9] formulates an optimization problem in the latent space and proposes a text-guided image restoration algorithm. Such LDM-based solvers have proven effective in constraining the solution space even for inverse problems at higher scales.

Despite such notable progress in LDM-based inverse problem solvers, they often prove less effective when extended to the current dominant trend—flow-based models [12, 14]. Flow-based models offer a robust foundation, and their integration into scalable architectures like the Diffusion Transformer has led to powerful generative capabilities

[6, 18]. Yet, only a few flow-based inverse problem solvers have been proposed [15, 16]. For example, FlowChef [16] guides the velocity field using the gradient of the conditioning loss with respect to clean estimates computed at each point of the flow ODE under the assumption of local linear vector field and locally constant Jacobian. PnP-Flow [15] uses a pre-trained flow model as the denoiser for a plug-and-play restoration algorithm. However such existing methods do not provide enough insight into how they are related to inverse problem solving using posterior sampling.

Therefore, one of the key contributions of this paper is providing a comprehensive guide to posterior sampling within the flow-based framework, addressing its subtleties and challenges. Building on these insights, we propose a novel approach that achieves unmatched reconstruction speed and quality, even for high-resolution images with severe degradations. Specifically, we introduce *Flow-Driven Posterior Sampling (FlowDPS)*, a novel flow-based inverse problem solver derived from the decomposition of the flow ODE. More specifically, by deriving the flow-version of Tweedie’s formula, we demonstrate that the flow ODE can be broken down into two components—one for clean image estimation and another for noise estimation—and that manipulating both components significantly enhances reconstruction quality. Under the decomposition, we can easily see that posterior sampling with data consistency is incorporated into the estimated clean image by integrating the likelihood gradient, while generative quality is maintained by adding stochastic noise to the noise component. Consequently, we can easily show that the geometric structure of flow ODE is indeed similar to that of the diffusion model. Furthermore, we show that FlowDPS can be easily integrated into the state-of-the-art latent flow models.

We validate FlowDPS on various linear inverse problems using the widely adopted flow-based Stable Diffusion 3.0 model [6]. Extensive experiments confirm that our method outperforms any existing LDM-based or flow-based solver. Overall, FlowDPS enables state-of-the-art reconstruction of high-resolution images for various inverse problems.

2. Background

Flow-based Models and Flow Matching Suppose that we have access to samples of source distribution p and target distribution q over $\mathbb{R}^d$. The goal of a flow-based model is to generate $X_0 \sim q$, starting from $X_1 \sim p$. Specifically, we define a time-dependent flow $\psi : (\mathbf{x}, t) \rightarrow \psi_t(\mathbf{x})$ such that $\psi_t(X_1) := X_t \sim p_t$, where $p_t(\mathbf{x})$ with $0 \leq t \leq 1$ denotes a probability path with boundary conditions $p_1 = p$ and $p_0 = q$ ¹. The flow can be uniquely defined by a flow

¹While conventional notation uses $p_0 = p$ and $p_1 = q$, we swap the time index to align with the implementation of the flow-based model.

ODE with velocity field v_t :

$$\frac{d\psi_t(\mathbf{x})}{dt} = v_t(\psi_t(\mathbf{x})), \quad \text{where } \psi_1(\mathbf{x}) = \mathbf{x} \quad (2)$$

Using the change of variable $\mathbf{x} = \psi_t^{-1}(\mathbf{x}')$, one can compute the corresponding velocity field $v_t(\mathbf{x}')$ by

$$v_t(\mathbf{x}') = v_t(\psi_t(\mathbf{x})) = \frac{d}{dt}\psi_t(\mathbf{x}) = \dot{\psi}_t(\psi_t^{-1}(\mathbf{x}')), \quad (3)$$

where $\dot{\psi}_t = d\psi_t/dt$. When we train the flow model as a generative model, v_t is learned using neural network parameterized by θ through flow matching loss [12]:

$$\min_{\theta} \mathbb{E}_{t, \mathbf{x} \sim p_t} \|v_t(\mathbf{x}_t) - v_t^{\theta}(\mathbf{x}_t)\|^2. \quad (4)$$

Unfortunately, a key problem to flow matching is that we cannot access $v_t(\mathbf{x}_t)$ due to intractable integration over all $\mathbf{x}_0$. To address this, [12] proposes conditional flow $\psi_t(\mathbf{x}|\mathbf{x}_0)$ that generate the probability $p_t(\mathbf{x}|\mathbf{x}_0)$. Using Eq. (3), the conditional velocity field can be computed as

$$v_t(\mathbf{x}|\mathbf{x}_0) = \dot{\psi}_t(\psi_t^{-1}(\mathbf{x}|\mathbf{x}_0)|\mathbf{x}_0), \quad (5)$$

and the parameterized velocity field is learned using the conditional flow matching loss,

$$\min_{\theta} \mathbb{E}_{t, \mathbf{x} \sim p_{t|0}(\mathbf{x}_t|\mathbf{x}_0)} \|v_t(\mathbf{x}_t|\mathbf{x}_0) - v_t^{\theta}(\mathbf{x}_t)\|^2, \quad (6)$$

whose gradient with respect to θ is shown to match that of Eq. (4) [12].

Affine conditional flows In practice, the flow model with affine coupling is one of the most widely used flow framework. Specifically, let the source and target distributions form an independent coupling $\pi_{0,1}(\mathbf{x}_0, \mathbf{x}_1) = q(\mathbf{x}_0)p(\mathbf{x}_1)$ where $p(\mathbf{x}_1) = \mathcal{N}(0, \mathbf{I})$. Now consider affine conditional flows:

$$\psi_t(\mathbf{x}|\mathbf{x}_0) = a_t\mathbf{x}_0 + b_t\mathbf{x} \quad (7)$$

with boundary conditions $a_0 = 1, b_0 = 0$ and $a_1 = 0, b_1 = 1$ such that $\psi_0(\mathbf{x}|\mathbf{x}_0) = \mathbf{x}_0$ and $\psi_1(\mathbf{x}|\mathbf{x}_0) = \mathbf{x}$. The forward path given condition $\mathbf{x}_0$ is then given by

$$\mathbf{x}_t = \psi_t(\mathbf{x}_1|\mathbf{x}_0) = a_t\mathbf{x}_0 + b_t\mathbf{x}_1 \quad (8)$$

Using Eq. (5) and Eq. (8), the conditional velocity at $\mathbf{x}_t$ can be obtained by

$$v_t(\mathbf{x}_t|\mathbf{x}_0) = \dot{\psi}_t(\mathbf{x}_1|\mathbf{x}_0) = \dot{a}_t\mathbf{x}_0 + \dot{b}_t\mathbf{x}_1. \quad (9)$$

For the case of linear conditional flow, where $a_t = 1 - t$ and $b_t = t$, the conditional velocity is then given by

$v_t(\mathbf{x}_t|\mathbf{x}_0) = \dot{\psi}_t(\mathbf{x}_t|\mathbf{x}_0) = \mathbf{x}_1 - \mathbf{x}_0$, which leads to the popular form of the conditional flow matching loss:

$$\min_{\theta} \mathbb{E}_{t, \mathbf{x} \sim p_{t|0}} \|v_t^{\theta}(\mathbf{x}_t) - (\mathbf{x}_1 - \mathbf{x}_0)\|^2, \quad (10)$$

where $\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1$. It is important to note that, despite utilizing conditional flow matching, the parameterized flow velocity estimate $v_t^{\theta}(\mathbf{x}_t)$ obtained through a neural network approximates the *marginal* velocity $v_t(\mathbf{x}_t)$ rather than the conditional velocity $v_t(\mathbf{x}_t|\mathbf{x}_0)$.

3. Main Contribution

3.1. Decomposition of Flow ODE

Mathematically, the marginal velocity field $v_t(\mathbf{x})$ can be computed by [12, 14]

$$\begin{aligned} v_t(\mathbf{x}) &= \mathbb{E}[v_t(\mathbf{x}_t|\mathbf{x}_0)|\mathbf{x}_t = \mathbf{x}] = \mathbb{E}[\dot{\psi}_t(\mathbf{x}_t|\mathbf{x}_0)|\mathbf{x}_t = \mathbf{x}] \\ &= \dot{a}_t\mathbb{E}[\mathbf{x}_0|\mathbf{x}_t = \mathbf{x}] + \dot{b}_t\mathbb{E}[\mathbf{x}_1|\mathbf{x}_t = \mathbf{x}], \end{aligned} \quad (11)$$

where we use $\dot{\psi}_t(\mathbf{x}_t|\mathbf{x}_0) = \dot{a}_t\mathbf{x}_0 + \dot{b}_t\mathbf{x}_1$. Here $\mathbb{E}[\mathbf{x}_0|\mathbf{x}_t]$ and $\mathbb{E}[\mathbf{x}_1|\mathbf{x}_t]$ corresponds to the denoised and noisy estimate for given $\mathbf{x}_t$, respectively, which can be computed using the following flow-version of Tweedie formula whose proof can be found in Appendix.

Proposition 1 (Tweedie Formula). *The denoised and noisy estimate given $\mathbf{x}_t$ are given by*

$$\begin{aligned} \mathbb{E}[\mathbf{x}_0|\mathbf{x}_t] &= \left[a_t - \dot{a}_t \frac{b_t}{\dot{b}_t} \right]^{-1} \left(\mathbf{x}_t - \frac{b_t}{\dot{b}_t} v_t(\mathbf{x}_t) \right) \\ \mathbb{E}[\mathbf{x}_1|\mathbf{x}_t] &= \left[b_t - \dot{b}_t \frac{a_t}{\dot{a}_t} \right]^{-1} \left(\mathbf{x}_t - \frac{a_t}{\dot{a}_t} v_t(\mathbf{x}_t) \right) \end{aligned}$$

This decomposition of the velocity through two Tweedie estimates leads to the decomposition of the flow ODE. Specifically, the flow ODE in Eq. (2) can be solved using the Euler method:

$$\mathbf{x}_{t+dt} = \mathbf{x}_t + v_t(\mathbf{x}_t)dt \quad (12)$$

$$= C_1(t)\hat{\mathbf{x}}_{0|t} + C_2(t)\hat{\mathbf{x}}_{1|t} \quad (13)$$

where $dt < 0$ denotes step size, $C_1(t) = a_t + \dot{a}_tdt$, $C_2(t) = b_t + \dot{b}_tdt$, $\hat{\mathbf{x}}_{0|t} := \mathbb{E}[\mathbf{x}_0|\mathbf{x}_t]$ and $\hat{\mathbf{x}}_{1|t} := \mathbb{E}[\mathbf{x}_1|\mathbf{x}_t]$. Note that the second term in Eq. (13) corresponds to deterministic noise in the literature of DDIM sampling. Inspired by the merits of stochastic noise in the reverse sampling process, we propose to mix stochastic noise as

$$\mathbf{x}_{t+dt} = C_1(t)\hat{\mathbf{x}}_{0|t} + C_2(t)\tilde{\mathbf{x}}_{1|t}, \quad (14)$$

where

$$\tilde{\mathbf{x}}_{1|t} = \sqrt{1 - \eta_t}\hat{\mathbf{x}}_{1|t} + \sqrt{\eta_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}). \quad (15)$$

As shown in Section 6, this leads to

$$\mathbf{x}_{t+dt} = C_1(t)\hat{\mathbf{x}}_{0|t} + \sqrt{C_2(t)^2 - k_t^2}\hat{\mathbf{x}}_{1|t} + \sqrt{k_t^2}\epsilon \quad (16)$$

which is the same form with **generalized version of DDIM** [23] with coefficients $C_1(t) = a_t + \dot{a}_t dt = a_{t+dt}$ and $C_2(t) = b_t + \dot{b}_t dt = b_{t+dt}$.

3.2. Posterior Sampling via Flow Models

Posterior flow velocity. Now we derive the flow velocity for a given posterior distribution. Using the Tweedie formula in Proposition 1, we get

$$\begin{aligned} v_t(\mathbf{x}_t) &= \frac{\dot{a}_t}{a_t}\mathbf{x}_t + \left[\dot{b}_t - b_t \frac{\dot{a}_t}{a_t} \right] \mathbb{E}[\mathbf{x}_1|\mathbf{x}_t] \\ &= \frac{\dot{a}_t}{a_t}\mathbf{x}_t - \left[\dot{b}_t b_t - b_t^2 \frac{\dot{a}_t}{a_t} \right] \nabla \log p_t(\mathbf{x}_t), \end{aligned} \quad (17)$$

where the second equality comes from [12]

$$\begin{aligned} \nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t) &= \mathbb{E}_{\mathbf{x}_0 \sim q} [\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t|\mathbf{x}_0)|\mathbf{x}_t] \\ &= -\frac{1}{b_t} \mathbb{E}[\mathbf{x}_1|\mathbf{x}_t] \end{aligned} \quad (18)$$

since $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|\mathbf{x}_0) = -\frac{1}{b_t^2}(\mathbf{x}_t - a_t\mathbf{x}_0)$ which arises from $p_t(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t|a_t\mathbf{x}_0, b_t^2\mathbf{I})$.

Given that $v_t(\mathbf{x}_t)$ in Eq. (17) generates the probability path $p_t(\mathbf{x}_t)$, we can easily see that for the given measurement model Eq. (1), the velocity field conditioned on measurement $\mathbf{y}$ given by

$$v_t(\mathbf{x}_t|\mathbf{y}) = \frac{\dot{a}_t}{a_t}\mathbf{x}_t - \left[\dot{a}_t b_t - b_t^2 \frac{\dot{a}_t}{a_t} \right] \nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t|\mathbf{y}). \quad (19)$$

generates the posterior sample path according to $p_t(\mathbf{x}_t|\mathbf{y})$. Using Bayes' rule, the score of the posterior distribution is

$$\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t|\mathbf{y}) = \nabla_{\mathbf{x}_t} \log p_t(\mathbf{y}|\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t). \quad (20)$$

Accordingly, the velocity field is expressed by [12]

$$v_t(\mathbf{x}_t|\mathbf{y}) = v_t(\mathbf{x}_t) - \zeta_t \nabla_{\mathbf{x}_t} \log p_t(\mathbf{y}|\mathbf{x}_t), \quad (21)$$

where $\zeta_t = \dot{a}_t b_t - b_t^2 \frac{\dot{a}_t}{a_t}$. Thus, we can conduct posterior sampling by adding likelihood gradient to the original velocity field.

Unfortunately, the calculation of $\nabla_{\mathbf{x}_t} \log p_t(\mathbf{y}|\mathbf{x}_t)$ is computationally prohibited due to the integration with respect to $\mathbf{x}_0$. Consequently, one could use the popular approximation technique in diffusion model called DPS [3]:

$$\nabla_{\mathbf{x}_t} \log p_t(\mathbf{y}|\mathbf{x}_t) \approx \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbb{E}[\mathbf{x}_0|\mathbf{x}_t]) \quad (22)$$

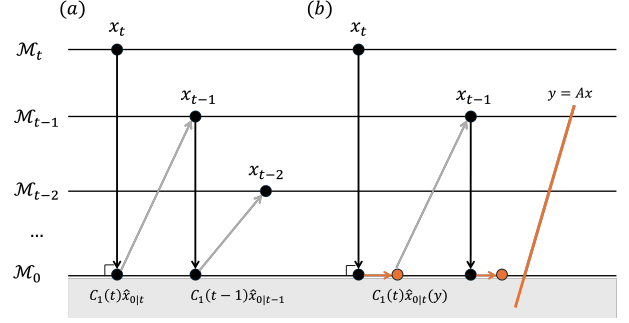


Figure 2. Geometry of FlowDPS. (a) Unconditional sampling of flow models where denoising and renoising are performed alternatively. (b) Posterior sampling of flow models where the data consistency offset is added to the denoised estimate. Orange arrow denotes the likelihood gradient in Eq. (35).

As Eq. (22) requires backpropagation through neural networks, which could be often unstable, one could use further approximation assuming piecewise linearity of the clean image manifold. Specifically, using the change of the variable, we have

$$\nabla_{\mathbf{x}_t} \log p_t(\mathbf{y}|\hat{\mathbf{x}}_{0|t}) = J_\theta(\mathbf{x}_t) \nabla_{\hat{\mathbf{x}}_{0|t}} \log p(\mathbf{y}|\hat{\mathbf{x}}_{0|t}), \quad (23)$$

where $\hat{\mathbf{x}}_{0|t} := \mathbb{E}[\mathbf{x}_0|\mathbf{x}_t]$ refers to the denoised estimate and $J_\theta(\mathbf{x}_t) = \partial \hat{\mathbf{x}}_{0|t} / \partial \mathbf{x}_t$ denotes the Jacobian. The following proposition, which is a flow-version inspired by [5], gives us an insight how to bypass Jacobian computation.

Proposition 2. Suppose the clean data manifold $\mathcal{M}$ is represented as an affine subspace and assume uniform distribution over $\mathcal{M}$. Then,

$$J_\theta(\mathbf{x}_t) = \frac{\partial \hat{\mathbf{x}}_{0|t}}{\partial \mathbf{x}_t} = \frac{1}{a_t} \mathcal{P}_{\mathcal{M}} \quad (24)$$

where $\mathcal{P}_{\mathcal{M}}$ denotes the orthogonal projection to $\mathcal{M}$.

This implies that if the standard gradient at the denoised estimate $\hat{\mathbf{x}}_{0|t}$ does not leave the clean manifold, we have

$$\begin{aligned} \nabla_{\mathbf{x}_t} \log p_t(\mathbf{y}|\hat{\mathbf{x}}_{0|t}) &= \frac{1}{a_t} \mathcal{P}_{\mathcal{M}} \nabla_{\hat{\mathbf{x}}_{0|t}} \log p(\mathbf{y}|\hat{\mathbf{x}}_{0|t}) \\ &= \frac{1}{a_t} \nabla_{\hat{\mathbf{x}}_{0|t}} \log p(\mathbf{y}|\hat{\mathbf{x}}_{0|t}) \end{aligned} \quad (25)$$

Hence, the resulting posterior velocity field is given by

$$v_t(\mathbf{x}_t|\mathbf{y}) \approx v_t(\mathbf{x}_t) - \frac{\zeta_t}{a_t} \nabla_{\hat{\mathbf{x}}_{0|t}} \log p(\mathbf{y}|\hat{\mathbf{x}}_{0|t}), \quad (26)$$

where $\frac{\zeta_t}{a_t} = \dot{a}_t \frac{b_t}{a_t} (1 - \frac{b_t}{a_t})$.

Posterior flow sampling. Finally, we solve the Euler method for flow ODE using the posterior flow velocity:

$$\begin{aligned} \mathbf{x}_{t+dt} &= \mathbf{x}_t + v_t(\mathbf{x}_t|\mathbf{y})dt \\ &= \mathbf{x}_t + \left(v_t(\mathbf{x}_t) - \frac{\zeta_t}{a_t} \nabla_{\hat{\mathbf{x}}_{0|t}} \log p(\mathbf{y}|\hat{\mathbf{x}}_{0|t}) \right) dt \\ &= C_1(t)\tilde{\mathbf{x}}_{0|t} + C_2(t)\tilde{\mathbf{x}}_{1|t}, \end{aligned} \quad (27)$$

where

$$\tilde{\mathbf{x}}_{0|t} = \hat{\mathbf{x}}_{0|t} - \beta_t \nabla_{\hat{\mathbf{x}}_{0|t}} \log p(\mathbf{y}|\hat{\mathbf{x}}_{0|t}) \quad (28)$$

with $\beta_t := \frac{\zeta_t}{a_t} \frac{dt}{C_1(t)}$. By inspecting Eq. (13) and Eq. (14), we can also use the following DDIM version of posterior flow in an SDE form:

$$\mathbf{x}_{t+dt} = C_1(t)\tilde{\mathbf{x}}_{0|t} + C_2(t)\tilde{\mathbf{x}}_{1|t}, \quad (29)$$

where $\tilde{\mathbf{x}}_{1|t}$ is given by Eq. (15). We call this general flow-based inverse problem solver as **Flow-Driven Posterior Sampling (FlowDPS)**. Fig. 2 depicts the geometric insight of FlowDPS, which shows striking similarity to the geometry of various diffusion posterior sampling methods.

Linear conditional flow. For the case of linear conditional flow, we have $a_t = 1 - t$ and $b_t = t$, leading to $C_1(t) = 1 - \sigma_{t+dt}$, $C_2(t) = \sigma_{t+dt}$, where $\sigma_t = t$. Furthermore, the corresponding Tweedie formula becomes

$$\hat{\mathbf{x}}_{0|t} = \mathbf{x}_t - tv_t(\mathbf{x}_t), \quad \hat{\mathbf{x}}_{1|t} = \mathbf{x}_t + (1 - t)v_t(\mathbf{x}_t) \quad (30)$$

Then, FlowDPS can be implemented as

$$\mathbf{x}_{t+dt} = (1 - \sigma_{t+dt})\tilde{\mathbf{x}}_{0|t} + \sigma_{t+dt}\tilde{\mathbf{x}}_{1|t}, \quad (31)$$

where $\tilde{\mathbf{x}}_{0|t}$ and $\tilde{\mathbf{x}}_{1|t}$ are from Eq. (28) and Eq. (15).

3.3. Comparison with other methods

In this section, we discuss the improvements of FlowDPS compared to existing methods. FlowChef [16] attempts to guide intermediate points of the flow ODE using the gradient of a loss function,

$$\mathbf{x}_t \leftarrow \mathbf{x}_t - s \nabla_{\mathbf{x}_{0|t}} \mathcal{L}(\mathbf{A}\hat{\mathbf{x}}_{0|t}, \mathbf{y}) \quad (32)$$

where $\mathcal{L}(\mathbf{A}\hat{\mathbf{x}}_{0|t}, \mathbf{y}) = \|\mathbf{A}\hat{\mathbf{x}}_{0|t} - \mathbf{y}\|^2$ and s denotes constant step size. For readers, Eq. (32) may seem similar to our FlowDPS at first glance. However, there are crucial differences between the two approaches.

While FlowChef applies a guiding gradient for $\mathbf{x}_t$ as a whole, FlowDPS decomposes the flow ODE into $\mathbf{x}_{0|t}$ and $\mathbf{x}_{1|t}$, applying the gradient to the appropriate component $\mathbf{x}_{0|t}$. Specifically, FlowDPS allows for inherent usage of adaptive step sizing with the scaling factor β_t , and incorporation of stochastic noise-enhancements, which cannot be trivially introduced within the FlowChef framework. As

illustrated in Fig. 6 in the Appendix 9.1, $-\beta_t$ rapidly decreases to zero. This progression implies that our algorithm prioritizes likelihood maximization in the early denoising steps, aligning the sample with measurements. Then, as denoising proceeds, the process gradually reverts to the unconstrained flow trajectory, compensating for any deviation introduced by the likelihood gradient. This adaptive weighting of the guidance term is crucial. Empirically, we found that the overall structure of the sample is primarily determined in the early stages of the flow, while fine details are refined in the later stages. Thus, by dynamically adjusting the guidance influence, FlowDPS ensures a smooth transition from a measurement-consistent solution to a generative prior-driven process. Most importantly, our approach establishes a clearer connection to posterior sampling, offering a more principled framework compared to prior methods [15, 16], which introduce a guiding vector to the ODE trajectory in a more heuristic manner.

3.4. Latent FlowDPS

Similar to the score-based diffusion models, flow-based models can be defined in latent space. This enables flexible conditioning and reduces memory/time cost for generation. Specifically, let $\mathcal{E}_\phi$ and $\mathcal{D}_\varphi$ as the encoder and decoder that map samples between image and latent spaces, satisfying

$$\mathbf{z}_0 = \mathcal{E}_\phi(\mathbf{x}_0) = \mathcal{E}_\phi(\mathcal{D}_\varphi(\mathbf{z}_0)) \quad (33)$$

where $\mathbf{x}_0$ and $\mathbf{z}_0$ denote clean image and corresponding latent code, respectively. As $\mathcal{E}_\phi$ and $\mathcal{D}_\varphi$ are fixed in our method, we discard subscript ϕ and φ for brevity in the following sections. By defining the target distribution p_0 as the distribution of latent code, conditional flow $\psi_t(\mathbf{z}|\mathbf{z}_0)$ and conditional velocity field $v_t(\mathbf{z}|\mathbf{z}_0)$ can be defined. This allows us to solve problems at higher pixel resolutions, to leverage text conditions for the solution, and to leverage foundational generative models (such as various versions of Stable Diffusion).

Specifically, the denoised estimate of the latent code $\mathbf{z}_t$ is expressed as

$$\tilde{\mathbf{z}}_{0|t} = \hat{\mathbf{z}}_{0|t} - \beta_t \nabla_{\hat{\mathbf{z}}_{0|t}} \log p(\mathbf{y}|\hat{\mathbf{z}}_{0|t}). \quad (34)$$

The gradient is computed as

$$\begin{aligned} \nabla_{\hat{\mathbf{z}}_{0|t}} \log p(\mathbf{y}|\hat{\mathbf{z}}_{0|t}) &= \nabla_{\hat{\mathbf{z}}_{0|t}} \log \int p(\mathbf{y}|\mathbf{x}_0)p(\mathbf{x}_0|\hat{\mathbf{z}}_{0|t})d\mathbf{x}_0 \\ &\simeq \nabla_{\hat{\mathbf{z}}_{0|t}} \log p(\mathbf{y}|\mathcal{D}(\hat{\mathbf{z}}_{0|t})) \end{aligned} \quad (35)$$

where the approximation comes from the assumption that $p(\mathbf{x}_0|\mathbf{z}_0) := \delta(\mathbf{x}_0 - \mathcal{D}(\mathbf{z}_0))$ with decoder $\mathcal{D}$ for a near-perfect autoencoder. As our forward model in Eq. (1) includes Gaussian noise, we can model

$$p(\mathbf{y}|\mathbf{x}_0) = \mathcal{N}(\mathbf{y}|\mathbf{A}\mathbf{x}_0, \sigma_n^2 \mathbf{I}_m) \quad (36)$$

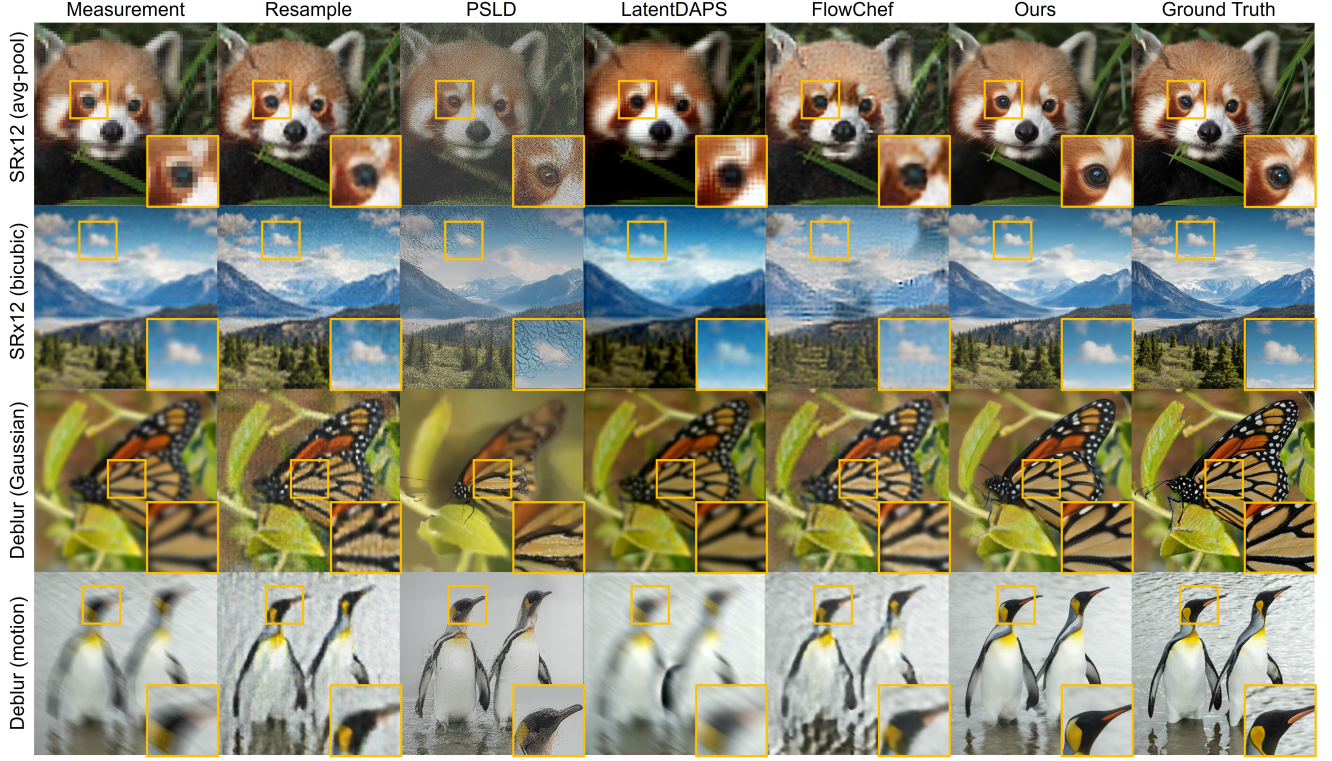


Figure 3. Qualitative comparison for linear inverse problems with DIV2K validation set. Insets show an enlarged view of the highlighted yellow boxes

and the likelihood gradient Eq. (35) is computed by

$$\nabla_{\hat{\mathbf{z}}_{0|t}} \log p(\mathbf{y}|\mathcal{D}(\hat{\mathbf{z}}_{0|t})) = -\nabla_{\hat{\mathbf{z}}_{0|t}} \frac{\|\mathbf{y} - \mathcal{AD}(\hat{\mathbf{z}}_{0|t})\|^2}{2\sigma_n^2}. \quad (37)$$

Building on the concept of Decomposed Diffusion Sampling (DDS) [3], which employs multiple conjugate gradient steps to refine the denoised estimate, thereby accelerating convergence to the inverse solution while preserving updates within the affine space, we adopt a similar approach. Specifically, instead of a single gradient step for $\hat{\mathbf{z}}_{0|t}$ as in Eq. (34), we perform multiple gradient steps to obtain $\hat{\mathbf{z}}_{0|t}(\mathbf{y})$, enhancing the accuracy of the estimate. To mitigate significant deviations of $\hat{\mathbf{z}}_{0|t}$ from the flow model’s sampling trajectory due to data consistency updates, we introduce an interpolation mechanism between the initial initial $\hat{\mathbf{z}}_{0|t}$ and updated $\hat{\mathbf{z}}_{0|t}(\mathbf{y})$. This is formulated as $\hat{\mathbf{z}}_{0|t} = (1 - \gamma)\hat{\mathbf{z}}_{0|t} + \gamma\hat{\mathbf{z}}_{0|t}(\mathbf{y})$. This interpolation ensures that the update remains aligned with the flow model’s trajectory, preventing excessive divergence while maintaining consistency with the observed data.

Specifically, we devise our algorithm by setting the values $\gamma = \sigma_t$ and $\eta = 1 - \sigma_{t+dt}$. This is to imitate the behavior of β_t in Eq. (28), which monotonically decreases

to zero. Specifically, the selection $\gamma = \sigma_t$ puts emphasis on $\hat{\mathbf{z}}_{0|t}(\mathbf{y})$ in early steps, guiding $\hat{\mathbf{z}}_{0|t}$ towards a more data-consistent path. $\hat{\mathbf{z}}_{0|t}$ is weighted more highly in later steps for better generation of high-frequency details. Similarly, stochastic noise ϵ is added under $\eta = 1 - \sigma_{t+dt}$ to use a higher level of stochastic noise in later steps and a higher level of deterministic noise $\hat{\mathbf{z}}_{1|t}$ in early steps. Further analysis regarding our selections is provided in Sec. 4.3. The overall algorithm is described in Algorithm 1.

4. Experiments

4.1. Experimental setup

Datasets and evaluation metrics. To demonstrate the performance of the proposed method in various domains, we use three datasets: 1k validation images of AFHQ, FFHQ, and 0.8k training images of DIV2K. We set the image resolution to 768×768 by resizing the original dataset. Quantitative evaluation is conducted on PSNR, SSIM for pixel-level fidelity, and FID, LPIPS for perceptual quality.

Baselines. We compare our method with PSLD [20], LatentDAPS [27], Resample [22] (recent latent diffusion-based inverse problem solvers), and FlowChef [16] (a conditioning framework designed for flow-based models). To ensure fair comparison, we use the pre-trained Stable Diffu-

AFHQ 1k (768 × 768)																
Super-resolution x12 (Avgpool)				Super-resolution x12 (Bicubic)				Deblurring (Gauss)				Deblurring (Motion)				
Method	PSNR↑	SSIM↑	FID↓	LPIPS↓	PSNR↑	SSIM↑	FID↓	LPIPS↓	PSNR↑	SSIM↑	FID↓	LPIPS↓	PSNR↑	SSIM↑	FID↓	LPIPS↓
ReSample	<u>24.92</u>	0.677	41.17	0.300	<u>24.94</u>	0.676	39.94	0.297	<u>24.67</u>	<u>0.665</u>	44.22	0.325	24.97	<u>0.688</u>	38.78	0.293
LatentDAPS	17.76	0.492	82.66	0.406	18.35	0.565	43.12	0.325	22.15	0.653	<u>27.89</u>	0.280	21.61	0.670	52.28	0.315
PSLD	15.66	0.249	81.42	0.490	15.23	0.274	138.6	0.521	13.87	0.476	240.6	0.548	13.63	0.479	257.9	0.563
FlowChef	23.93	0.641	<u>21.14</u>	<u>0.249</u>	24.79	0.633	<u>21.31</u>	<u>0.256</u>	22.75	0.703	36.46	<u>0.267</u>	26.16	0.732	39.19	<u>0.246</u>
Ours	25.88	<u>0.661</u>	16.85	0.198	26.07	<u>0.662</u>	15.71	0.188	25.33	0.653	23.00	0.238	<u>25.61</u>	0.657	19.99	0.222
FFHQ 1k (768 × 768)																
Super-resolution x12 (Avgpool)				Super-resolution x12 (Bicubic)				Deblurring (Gauss)				Deblurring (Motion)				
Method	PSNR↑	SSIM↑	FID↓	LPIPS↓	PSNR↑	SSIM↑	FID↓	LPIPS↓	PSNR↑	SSIM↑	FID↓	LPIPS↓	PSNR↑	SSIM↑	FID↓	LPIPS↓
ReSample	25.09	0.734	102.7	0.304	25.09	0.732	102.4	0.301	<u>24.95</u>	<u>0.724</u>	107.4	0.333	<u>25.39</u>	<u>0.745</u>	95.16	0.303
LatentDAPS	18.09	0.541	128.8	0.368	18.62	0.614	105.1	0.294	22.56	0.714	<u>72.97</u>	<u>0.248</u>	22.06	<u>0.722</u>	<u>81.63</u>	0.287
PSLD	15.05	0.240	142.1	0.471	15.20	0.275	141.6	0.472	12.98	0.519	265.6	0.563	12.98	0.527	268.1	0.575
FlowChef	<u>25.40</u>	0.696	<u>41.50</u>	<u>0.218</u>	<u>25.31</u>	0.692	<u>39.75</u>	<u>0.224</u>	23.14	0.767	112.8	0.251	23.14	0.772	104.7	<u>0.264</u>
Ours	26.81	<u>0.703</u>	33.78	0.159	27.00	<u>0.702</u>	33.75	0.154	26.23	0.702	41.31	0.197	26.62	0.699	38.14	0.182
DIV2K 0.8k (768 × 768)																
Super-resolution x12 (Avgpool)				Super-resolution x12 (Bicubic)				Deblurring (Gauss)				Deblurring (Motion)				
Method	PSNR↑	SSIM↑	FID↓	LPIPS↓	PSNR↑	SSIM↑	FID↓	LPIPS↓	PSNR↑	SSIM↑	FID↓	LPIPS↓	PSNR↑	SSIM↑	FID↓	LPIPS↓
ReSample	<u>20.41</u>	0.505	<u>101.5</u>	<u>0.352</u>	<u>20.48</u>	0.504	<u>99.73</u>	<u>0.349</u>	<u>20.31</u>	0.493	113.7	0.386	<u>20.82</u>	0.530	96.73	0.343
LatentDAPS	14.34	0.356	183.3	0.412	16.92	0.437	104.7	0.383	19.38	<u>0.503</u>	<u>83.72</u>	0.346	19.20	<u>0.542</u>	121.7	0.380
PSLD	13.90	0.187	124.0	0.523	14.01	0.207	118.1	0.519	14.92	0.313	97.20	0.527	14.54	0.307	108.3	0.547
FlowChef	18.02	<u>0.495</u>	<u>101.5</u>	0.355	15.12	0.441	123.4	0.433	19.54	0.520	98.27	<u>0.343</u>	19.80	0.546	<u>88.86</u>	<u>0.313</u>
Ours	20.84	0.488	58.29	0.258	21.10	<u>0.492</u>	56.19	0.252	20.46	0.473	72.13	0.319	21.04	0.495	62.68	0.291

Table 1. Quantitative comparison for linear inverse problems. For each dataset, we evaluate inverse problem solvers using 1k, 1k, and 0.8k image sets of AFHQ, FFHQ and DIV2K. **Bold**: the best, Underline: the second best.

Algorithm 1 Algorithm of FlowDPS (SD3.0, FLUX)

Require: Measurement $\mathbf{y}$, Linear operator $\mathbf{A}$, Pre-trained flow-based model $\mathbf{v}_\theta$, VAE encoder and Decoder $\mathcal{E}, \mathcal{D}$, Text embeddings $\mathbf{c}_\emptyset, \mathbf{c}$, CFG scale λ , Stochasticity level η , **Noise Schedule** σ_t

```

1:  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ 
2: for  $t : 1 \rightarrow 0$  do
3:    $\mathbf{v}_t(\mathbf{z}) \leftarrow \mathbf{v}_\theta(\mathbf{z}, \mathbf{c}_\emptyset) + \lambda(\mathbf{v}_\theta(\mathbf{z}, \mathbf{c}) - \mathbf{v}_\theta(\mathbf{z}, \mathbf{c}_\emptyset))$ 
4:    $\hat{\mathbf{z}}_{0|t} \leftarrow \mathbf{z} - \sigma_t \mathbf{v}_t(\mathbf{z})$ 
5:    $\hat{\mathbf{z}}_{1|t} \leftarrow \mathbf{z} + (1 - \sigma_t) \mathbf{v}_t(\mathbf{z})$ 
6:    $\triangleright 1.$  Likelihood Gradient
7:    $\hat{\mathbf{z}}_{0|t}(\mathbf{y}) \leftarrow \arg \min_{\mathbf{z}} \|\mathbf{y} - \mathbf{A}\mathcal{D}(\mathbf{z})\|^2$ 
8:    $\tilde{\mathbf{z}}_{0|t} \leftarrow \sigma_t \hat{\mathbf{z}}_{0|t}(\mathbf{y}) + (1 - \sigma_t) \hat{\mathbf{z}}_{0|t}$ 
9:    $\triangleright 2.$  Stochasticity
10:   $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ 
11:   $\tilde{\mathbf{z}}_{1|t} \leftarrow \sqrt{\sigma_{t+dt}} \tilde{\mathbf{z}}_{1|t} + \sqrt{1 - \sigma_{t+dt}} \epsilon$ 
12:   $\triangleright 3.$  Euler update
13:   $\mathbf{z} \leftarrow (1 - \sigma_{t+dt}) \tilde{\mathbf{z}}_{0|t} + \sigma_{t+dt} \tilde{\mathbf{z}}_{1|t}$ 
14: end for
```

sion 3.0 - medium [6] as the backbone model for all solvers. Further details are provided in the Appendix. The results of FLUX version of FlowDPS can be also found in Appendix.

Text conditioning. We leverage text prompts as conditions for the latent flow-based model to refine the solution. Specifically, we use the prompt “a photo of a closed face of a dog (cat)” for AFHQ, and the prompt “a photo of a closed face” for FFHQ. **Notably, defining an appropriate text prompt solely based on the given measurement can be**

Super-resolution x12 (Avgpool)					Super-resolution x12 (Bicubic)			
AFHQ	PSNR↑	SSIM↑	FID↓	LPIPS↓	PSNR↑	SSIM↑	FID↓	LPIPS↓
Ours (w/o ϵ)	24.75	0.609	32.85	0.251	24.80	0.602	34.81	0.268
Ours (w/ ϵ)	25.88	0.661	16.85	0.198	26.07	0.662	15.71	0.188
FFHQ	PSNR↑	SSIM↑	FID↓	LPIPS↓	PSNR↑	SSIM↑	FID↓	LPIPS↓
Ours (w/o ϵ)	25.61	0.668	78.67	0.249	25.53	0.656	82.07	0.275
Ours (w/ ϵ)	26.81	0.703	33.78	0.159	27.00	0.702	33.75	0.154

Table 2. Ablation of stochastic noise on AFHQ and FFHQ images.

challenging in practical scenarios. For instance, the text prompts for the DIV2K dataset must include more details compared to AFHQ or FFHQ, since each image may contain multiple objects. To evaluate effectiveness in such real-world scenarios, we utilize text prompts extracted from the measurement using **DAPE** [26] as a degradation-aware text prompt extractor. For all cases, we set CFG [7] scale to 2.0.

4.2. Experimental Results with Inverse Problems

Noisy linear inverse problems. FlowDPS is a general inverse problem solver that does not require additional training, thus we demonstrate its broad applicability across four linear inverse problems. Specifically, we consider i) Super-resolution from average pooling with scale factor of 12, ii) Super-resolution from bicubic interpolation with scale factor of 12, iii) Gaussian deblurring with kernel size of 61 and standard deviation of 3.0, and iv) Motion deblurring with kernel size of 61 and intensity value of 0.5. In all tasks, we add Gaussian noise to the measurement with $\sigma_n = 0.03$.

Comparison results. Quantitative and qualitative comparison results are shown in Table 1 and Figure 3. Results show that FlowDPS outperforms existing methods across all datasets for various linear inverse problems. In particu-

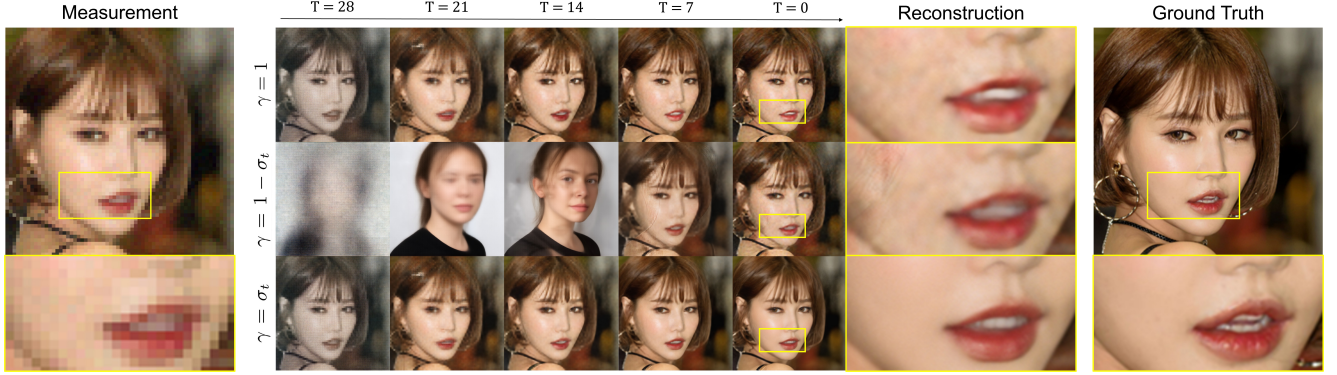


Figure 4. Sampling trajectories for various choices of interpolation scale γ . Insets show an enlarged view of the highlighted yellow boxes.

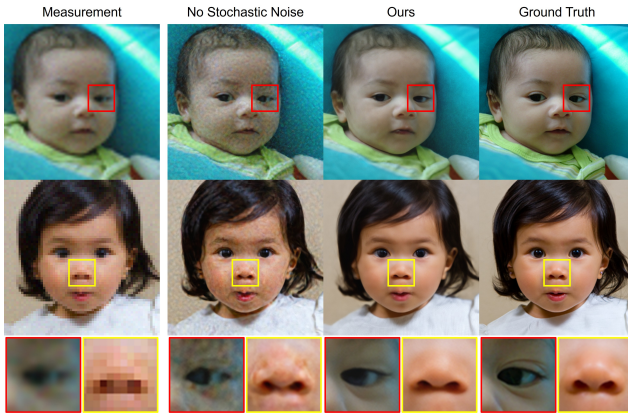


Figure 5. Qualitative results regarding ablation of stochastic noise.

lar, inverse problem solvers designed for latent diffusion-based models (*i.e.*, ReSample, LatentDAPS, PSLD) perform suboptimally when applied to the linear flow-based Stable Diffusion 3.0. These methods yield images that partially resolve the inverse problem, but their outputs remain significantly degraded—an anticipated challenge when reconstructing highly degraded, high-resolution (768×768) images. Thus, blurry results cause pixel-fidelity results (*e.g.*, SSIM) of baseline methods to be abnormally high in some cases as reported in [11, 28], but substandard perceptual results (*e.g.*, FID, LPIPS) signify the suboptimal performance. Consequently, inverse problem solvers devised for LDMs cannot be naively applied in flow-based settings, supporting the pressing need for a suitable framework.

In contrast to LDM-based solvers, methods specially developed for linear flow-based models (*i.e.*, FlowChef, FlowDPS) yield significantly improved performance regarding the metrics evaluated in Table 1. However, qualitative results show how FlowChef fails to achieve acceptable visual quality in challenging degradation scenarios. In contrast, FlowDPS consistently produces state-of-the-art results, regarding both image fidelity and perceptual quality.

More qualitative results, including higher resolutions and FLUX [10] implementation are provided in Appendix 11.

4.3. Ablation study

To further validate the effectiveness of our method, we perform various ablation studies regarding important selections: interpolation parameter γ and amount of stochastic noise η . Also, we explore the effect of NFE and CFG scale.

Interpolation parameter γ . In FlowDPS, we manipulate the clean image estimation to balance data consistency and sample quality. Specifically, the initial $\hat{z}_{0|t}$ and updated $\hat{z}_{0|t}(\mathbf{y})$ is interpolated based on an interpolation scale γ such that $\hat{z}_{0|t} = (1 - \gamma)\hat{z}_{0|t} + \gamma\hat{z}_{0|t}(\mathbf{y})$. Note that choice of parameter γ is flexible, and in Figure 4 we demonstrate the sampling trajectories of certain examples. Selecting $\gamma = 1$ (*i.e.*, using the updated $\hat{z}_{0|t}(\mathbf{y})$ directly) causes deviation of $\hat{z}_{0|t}$ from the sampling trajectory of the base model, causing reconstructions to become blurry. Selecting $\gamma = 1 - \sigma_t$ puts emphasis on $\hat{z}_{0|t}$ in the early stages of denoising and $\hat{z}_{0|t}(\mathbf{y})$ in the later stages, confusing the model in which sampling trajectory it should take. Since overall structure is determined early on in the denoising process (see Figure 4), it is reasonable to weight $\hat{z}_{0|t}(\mathbf{y})$ highly in the initial steps to generate data-consistent structure, and weight $\hat{z}_{0|t}$ highly in the final steps to better produce high-frequency fine details. Thus, in our final algorithm we select $\gamma = \sigma_t$.

Importance of stochastic noise. We provide quantitative and qualitative analysis regarding the importance of stochastic noise in Figure 5 and Table 2. Specifically, we observe that using only deterministic noise causes artifacts to appear, and that mixing stochastic noise removes such artifacts while guiding samples towards high-quality results.

Relation between NFE and CFG scale λ . NFE and CFG scale λ are important factors for generating high-quality results that contain high-frequency details. We find using NFE = 28 and $\lambda = 2.0$ to be sufficient for producing high-quality results. Further discussion regarding the relation between NFE and CFG scale is provided in Appendix 9.2.

5. Conclusion

This work addresses the pressing need for a general flow-based inverse problem solver, bridging a gap that LDM-based approaches have yet to fill. In order to draw on the powerful generative capacity of flow-based models, we presented Flow-Driven Posterior Sampling (FlowDPS), which carefully decomposes the flow ODE into clean image and noise components. Through targeted manipulation of each component, FlowDPS seamlessly integrates data consistency and preserves generative quality to yield superior results on various linear inverse problems even for high-resolution images with severe degradations. More broadly, the general framework we introduce clarifies the process of posterior sampling in flow-based generative models, offering both a theoretical perspective and a practical method. We anticipate that this approach will inspire further work across a wide range of applications.

References

- [1] Stephen Boyd, Neal Parikh, and Eric Chu. *Distributed optimization and statistical learning via the alternating direction method of multipliers*. Now Publishers Inc, 2011. 2
- [2] Hyungjin Chung, Byeongsu Sim, Dohoon Ryu, and Jong Chul Ye. Improving diffusion models for inverse problems using manifold constraints. In *Advances in Neural Information Processing Systems*, 2022. 2
- [3] Hyungjin Chung, Jeongsol Kim, Michael Thompson McCann, Marc Louis Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. In *International Conference on Learning Representations*, 2023. 2, 4, 6
- [4] Hyungjin Chung, Jong Chul Ye, Peyman Milanfar, and Mauricio Delbracio. Prompt-tuning latent diffusion models for inverse problems. *arXiv preprint arXiv:2310.01110*, 2023. 2
- [5] Hyungjin Chung, Suhyeon Lee, and Jong Chul Ye. Decomposed diffusion sampler for accelerating large-scale inverse problems. In *The Twelfth International Conference on Learning Representations*, 2024. 4
- [6] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024. 2, 7
- [7] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021. 7
- [8] Jeongsol Kim, Geon Yeong Park, Hyungjin Chung, and Jong Chul Ye. Regularization by texts for latent diffusion inverse solvers. *arXiv preprint arXiv:2311.15658*, 2023. 2
- [9] Jeongsol Kim, Geon Yeong Park, and Jong Chul Ye. Dream-sampler: Unifying diffusion sampling and score distillation for image manipulation. *arXiv preprint arXiv:2403.11415*, 2024. 2
- [10] Black Forest Labs. Flux. <https://github.com/black-forest-labs/flux>, 2024. 8, 4
- [11] Christian Ledig, Lucas Theis, Ferenc Huszar, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, et al. Photo-realistic single image super-resolution using a generative adversarial network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4681–4690, 2017. 8
- [12] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2023. 2, 3, 4
- [13] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, pages 34892–34916. Curran Associates, Inc., 2023. 4
- [14] Xingchao Liu, Chengyue Gong, and qiang liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2023. 2, 3
- [15] Ségolène Tiffany Martin, Anne Gagneux, Paul Hagemann, and Gabriele Steidl. Pnp-flow: Plug-and-play image restoration with flow matching. In *The Thirteenth International Conference on Learning Representations*, 2025. 2, 5
- [16] Maitreya Patel, Song Wen, Dimitris N Metaxas, and Yezhou Yang. Steering rectified flow models in the vector field for controlled image generation. *arXiv preprint arXiv:2412.00100*, 2024. 2, 5, 6
- [17] Suraj Patil, Pedro Cuenca, Nathan Lambert, and Patrick von Platen. Stable diffusion with diffusers. *Hugging Face Blog*, 2022. https://huggingface.co/blog/stable_diffusion. 2
- [18] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4195–4205, 2023. 2
- [19] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022. 2
- [20] Litu Rout, Negin Raoof, Giannis Daras, Constantine Caramanis, Alex Dimakis, and Sanjay Shakkottai. Solving linear inverse problems provably via posterior sampling with latent diffusion models. *Advances in Neural Information Processing Systems*, 36, 2024. 2, 6
- [21] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, et al. Laion-5b: An open large-scale dataset for training next generation image-text models. *Advances in Neural Information Processing Systems*, 35:25278–25294, 2022. 2
- [22] Bowen Song, Soo Min Kwon, Zecheng Zhang, Xinyu Hu, Qing Qu, and Liyue Shen. Solving inverse problems with latent diffusion models via hard data consistency. In *The Twelfth International Conference on Learning Representations*, 2024. 2, 6

- [23] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *9th International Conference on Learning Representations, ICLR*, 2021. [4](#), [1](#)
- [24] Jiaming Song, Arash Vahdat, Morteza Mardani, and Jan Kautz. Pseudoinverse-guided diffusion models for inverse problems. In *International Conference on Learning Representations*, 2023. [2](#)
- [25] Yinhuai Wang, Jiwen Yu, and Jian Zhang. Zero-shot image restoration using denoising diffusion null-space model. In *The Eleventh International Conference on Learning Representations*, 2023. [2](#)
- [26] Rongyuan Wu, Tao Yang, Lingchen Sun, Zhengqiang Zhang, Shuai Li, and Lei Zhang. Seesr: Towards semantics-aware real-world image super-resolution. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 25456–25467, 2024. [7](#), [4](#)
- [27] Bingliang Zhang, Wenda Chu, Julius Berner, Chenlin Meng, Anima Anandkumar, and Yang Song. Improving diffusion inverse problem solving with decoupled noise annealing. *arXiv preprint arXiv:2407.01521*, 2024. [2](#), [6](#)
- [28] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018. [8](#)

FlowDPS : Flow-Driven Posterior Sampling for Inverse Problems

Supplementary Material

6. Derivation of DDIM form

The stochastic part of the equation Eq. (14) is

$$\begin{aligned} C_2(t)\tilde{\mathbf{x}}_{1|t} &= C_2(t)(\sqrt{1-\eta_t}\hat{\mathbf{x}}_{1|t} + \sqrt{\eta_t}\epsilon) \\ &= \sqrt{C_2(t)^2 - C_2(t)^2\eta_t}\hat{\mathbf{x}}_{1|t} + \sqrt{C_2(t)^2\eta_t}\epsilon. \end{aligned} \quad (38)$$

If we set $k_t = C_2(t)\sqrt{\eta_t}$, this is equivalent to

$$\sqrt{C_2(t)^2 - k_t^2}\hat{\mathbf{x}}_{1|t} + \sqrt{k_t^2}\epsilon \quad (39)$$

and the one step update is expressed as

$$\begin{aligned} \mathbf{x}_{t+dt} &= C_1(t)\hat{\mathbf{x}}_{0|t} + \sqrt{C_2(t)^2 - k_t^2}\hat{\mathbf{x}}_{1|t} + \sqrt{k_t^2}\epsilon \end{aligned} \quad (40)$$

which is in the same form as DDIM [23] but with different coefficients $C_1(t) = a_t + \dot{a}_t dt = a_{t+dt}$ and $C_2(t) = b_t + \dot{b}_t dt = b_{t+dt}$.

7. Proofs

Proposition 1 (Tweedie Formula). *The denoised and noisy estimate given $\mathbf{x}_t$ are given by*

$$\begin{aligned} \mathbb{E}[\mathbf{x}_0|\mathbf{x}_t] &= \left[a_t - \dot{a}_t \frac{b_t}{\dot{b}_t} \right]^{-1} \left(\mathbf{x}_t - \frac{b_t}{\dot{b}_t} v_t(\mathbf{x}_t) \right) \\ \mathbb{E}[\mathbf{x}_1|\mathbf{x}_t] &= \left[b_t - \dot{b}_t \frac{a_t}{\dot{a}_t} \right]^{-1} \left(\mathbf{x}_t - \frac{a_t}{\dot{a}_t} v_t(\mathbf{x}_t) \right) \end{aligned}$$

Proof. According to Eq. (9), we have $v_t(\mathbf{x}_t|\mathbf{x}_0) = \dot{a}_t\mathbf{x}_0 + \dot{b}_t\mathbf{x}_1$. This leads to the representation of $\mathbf{x}_t$ with respect to the conditional flow velocity:

$$\begin{aligned} \mathbf{x}_t &= a_t\mathbf{x}_0 + b_t\mathbf{x}_1 \\ &= a_t\mathbf{x}_0 + b_t \left(\frac{v_t(\mathbf{x}_t|\mathbf{x}_0) - \dot{a}_t\mathbf{x}_0}{\dot{b}_t} \right) \\ &= \left(a_t - \dot{a}_t \frac{b_t}{\dot{b}_t} \right) \mathbf{x}_0 + \frac{b_t}{\dot{b}_t} v_t(\mathbf{x}_t|\mathbf{x}_0) \end{aligned} \quad (41)$$

Thus we have

$$\begin{aligned} \mathbb{E}[\mathbf{x}_0|\mathbf{x}_t] &= \left(a_t - \dot{a}_t \frac{b_t}{\dot{b}_t} \right)^{-1} \left(\mathbf{x}_t - \frac{b_t}{\dot{b}_t} \mathbb{E}[v_t(\mathbf{x}_t|\mathbf{x}_0)] \right) \\ &= \left(a_t - \dot{a}_t \frac{b_t}{\dot{b}_t} \right)^{-1} \left(\mathbf{x}_t - \frac{b_t}{\dot{b}_t} v_t(\mathbf{x}_t) \right) \end{aligned}$$

Similarly, we have

$$\begin{aligned} \mathbf{x}_t &= a_t\mathbf{x}_0 + b_t\mathbf{x}_1 \\ &= a_t \left(\frac{v_t(\mathbf{x}_t|\mathbf{x}_0) - \dot{b}_t\mathbf{x}_1}{\dot{a}_t} \right) + b_t\mathbf{x}_1 \\ &= \left(b_t - \dot{b}_t \frac{a_t}{\dot{a}_t} \right) \mathbf{x}_1 + \frac{a_t}{\dot{a}_t} v_t(\mathbf{x}_t|\mathbf{x}_0) \end{aligned} \quad (42)$$

which leads to

$$\begin{aligned} \mathbb{E}[\mathbf{x}_1|\mathbf{x}_t] &= \left(b_t - \dot{b}_t \frac{a_t}{\dot{a}_t} \right)^{-1} \left(\mathbf{x}_t - \frac{a_t}{\dot{a}_t} \mathbb{E}[v_t(\mathbf{x}_t|\mathbf{x}_0)] \right) \\ &= \left(b_t - \dot{b}_t \frac{a_t}{\dot{a}_t} \right)^{-1} \left(\mathbf{x}_t - \frac{a_t}{\dot{a}_t} v_t(\mathbf{x}_t) \right) \end{aligned}$$

□

Proposition 2. *Suppose the clean data manifold $\mathcal{M}$ is represented as an affine subspace and assume uniform distribution over $\mathcal{M}$. Then,*

$$J_\theta(\mathbf{x}_t) = \frac{\partial \hat{\mathbf{x}}_{0|t}}{\partial \mathbf{x}_t} = \frac{1}{a_t} \mathcal{P}_\mathcal{M} \quad (24)$$

where $\mathcal{P}_\mathcal{M}$ denotes the orthogonal projection to $\mathcal{M}$.

Proof. Using Eq. (11) and Eq. (18), we can express the Jacobian as

$$J_\theta(\mathbf{x}) = \frac{1}{a_t} \frac{\partial(\mathbf{x}_t - b_t^2 \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t))}{\partial \mathbf{x}_t}. \quad (43)$$

Here, we will derive the score function $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$ in a closed form solution with assumptions and prove the result.

From the definition of affine conditional flow,

$$\mathbf{x}_t = \psi_t(\mathbf{x}_1|\mathbf{x}_0) = a_t\mathbf{x}_0 + b_t\mathbf{x}_1, \quad (44)$$

where $\mathbf{x}_1 \sim \mathcal{N}(0, \mathbf{I}_d)$, we get the explicit expression of

$$p(\mathbf{x}_t|\mathbf{x}_0) = \frac{1}{(2\pi b_t^2)^{d/2}} \exp \left(-\frac{\|\mathbf{x}_t - a_t\mathbf{x}_0\|^2}{2b_t^2} \right). \quad (45)$$

Assume that the clean images are distribution on subspace $\mathcal{M}$ uniformly. To express this, we start from defining $p(\mathbf{x}_0)$ as a zero-mean Gaussian distribution with isotropic variance σ .

$$p(\mathbf{x}_0) = \frac{1}{(2\pi\sigma^2)^{l/2}} \exp \left(-\frac{\|\mathcal{P}_\mathcal{M}\mathbf{x}_0\|^2}{2\sigma^2} \right) \quad (46)$$

as $\mathcal{P}_M \mathbf{x}_0 = \mathbf{x}_0$. Considering the marginal density

$$p(\mathbf{x}_t) = \int p(\mathbf{x}_t | \mathbf{x}_0) p(\mathbf{x}_0) d\mathbf{x}_0, \quad (47)$$

we have to compute $p(\mathbf{x}_t | \mathbf{x}_0) p(\mathbf{x}_0)$ that is

$$p(\mathbf{x}_t | \mathbf{x}_0) p(\mathbf{x}_0) = \frac{1}{(2\pi b_t^2)^{d/2} (2\pi\sigma^2)^{l/2}} \exp(-d(\mathbf{x}_t, \mathbf{x}_0)), \quad (48)$$

where

$$\begin{aligned} d(\mathbf{x}_t, \mathbf{x}_0) &= \frac{\|\mathbf{x}_t - a_t \mathbf{x}_0\|^2}{2b_t^2} + \frac{\|\mathcal{P}_M \mathbf{x}_0\|^2}{2\sigma^2} \\ &= \frac{\|\mathcal{P}_M^\perp \mathbf{x}_t\|^2}{2b_t^2} + \frac{\|\mathcal{P}_M \mathbf{x}_t - a_t \mathbf{x}_0\|^2}{2b_t^2} + \frac{\|\mathcal{P}_M \mathbf{x}_0\|^2}{2\sigma^2} \\ &= \frac{\|\mathcal{P}_M^\perp \mathbf{x}_t\|^2 - c_t \|\mathcal{P}_M \mathbf{x}_t\|^2}{2b_t^2} + \frac{\|\mathcal{P}_M \mathbf{x}_0 - \frac{1-c_t}{a_t} \mathcal{P}_M \mathbf{x}_t\|^2}{\sigma^2 c_t}, \end{aligned}$$

and

$$c_t = \frac{b_t^2}{b_t^2 + \sigma^2 + a_t^2}. \quad (49)$$

Therefore, after integrating out with respect to $\mathbf{x}_0$, we have

$$\log p(\mathbf{x}_t) = -\frac{\|\mathcal{P}_M^\perp \mathbf{x}_t\|^2 - c_t \|\mathcal{P}_M \mathbf{x}_t\|^2}{2b_t^2} + \text{const.}, \quad (50)$$

leading to

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) = -\frac{\mathcal{P}_M^\perp \mathbf{x}_t - c_t \mathcal{P}_M \mathbf{x}_t}{b_t^2}. \quad (51)$$

By the assumption of uniform distribution in Eq. (46) with $\sigma \rightarrow \infty$, we have $c_t \rightarrow 0$. Therefore,

$$\lim_{\sigma \rightarrow \infty} \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) = -\frac{1}{b_t^2} \mathcal{P}_M^\perp \mathbf{x}_t \quad (52)$$

and we conclude that

$$J_\theta(\mathbf{x}_t) = \frac{1}{a_t} \frac{\partial(\mathbf{x}_t - b_t^2 \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t))}{\partial \mathbf{x}_t} \quad (53)$$

$$= \frac{1}{a_t} \frac{\partial(\mathbf{x}_t - \mathcal{P}_M^\perp \mathbf{x}_t)}{\partial \mathbf{x}_t} \quad (54)$$

$$= \frac{1}{a_t} \frac{\partial \mathcal{P}_M \mathbf{x}_t}{\partial \mathbf{x}_t} = \frac{1}{a_t} \mathcal{P}_M. \quad (55)$$

□

8. Implementation details

In this section, we provide implementation details of the FlowDPS and baselines. The code will be released to public at <https://github.com/FlowDPS-Inverse/FlowDPS>.

Implementation with flow models For a fair comparison, we re-implement baselines that are proposed with score-based diffusion models. In the following, we provide details for each implementation.

PSLD [20] extends DPS [3] to latent diffusion models by introducing a novel regularization loss for the autoencoder. Since its gradient is computed with respect to the intermediate sample $\mathbf{x}_t$ during the reverse diffusion process, the same algorithm can be implemented using Euler’s method without loss of generality.

ReSample incorporates data consistency into the reverse sampling process of LDMs by solving an optimization problem on some time steps. The key ideas of this framework is Stochastic Resampling, for renoising the optimized latent, and hard data consistency. We implement Resample for linear flow-based models by setting $\bar{\alpha}_t = \frac{(1-\sigma_t)^2}{\sigma_t^2 + (1-\sigma_t)^2}$. Resample uses various techniques such as dividing the sampling process into three stages and separately using soft data consistency and hard data consistency. We find that adopting this same setting for our comparison produces extremely poor results, primarily because of the small number of ODE steps of the Euler solver. We empirically find that using skip step size of 1 to perform hard data consistency on all steps produces best results, and use this setting for comparison.

LatentDAPS proposes a noise annealing process to decouple consecutive samples in a sampling trajectory, which enables solvers to create errors made in earlier steps. We implement LatentDAPS from the official code, with modification of the solving process to use the linear flow-based backbone model StableDiffusion 3.0 [6] and Euler solver.

Hyper-parameter setting For all implementations, we use StableDiffusion 3.0 [6] as our baseline model. Also, we set the shift factor of time scheduler to 4.0.

- **PSLD** We set $\eta = 1.0$ and $\gamma = 0.1$ by following the original paper setting, and use 200 NFEs as in [8].
- **Resample** We use the same resampling hyperparameter $\gamma \left(\frac{1-\bar{\alpha}_{t-1}}{\bar{\alpha}_t} \right) \left(1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}} \right)$ with $\gamma = 40$ as proposed in the original paper, reparameterizing $\bar{\alpha}_t$ as previously explained. Skip step size is set to 1 to perform sufficient hard data consistency steps.
- **LatentDAPS** We use ODE solver steps $N_{ODE} = 5$ and annealing scheduler $N_A = 28$, resulting in a total NFE of 120. Total step number N in Langevin Dynamics is set to 50, following the settings of the original paper [27].
- **FlowChef** For a constant step size for the FlowChef, we find the best configuration by grid search with 100 images. In consequence, we set the step size to 200 for the super-resolution tasks and 50 for the deblurring tasks.
- **FlowDPS** For data consistency optimization, we use 3 steps of gradient descent with step size 15 for all tasks.

9. Ablation Study

9.1. Analysis on β_t and γ_t

In FlowDPS, we interpolate $\hat{z}_{0|t}$ and $\hat{z}_{0|t}(\mathbf{y})$ with coefficient γ_t to ensure the data consistency update does not lead to excessive divergence for the flow model’s trajectory. For the selection of $\gamma_t = \sigma_t$, we refer to the progression of the adaptive step size β_t in Eq. (28). Specifically, our likelihood gradient is applied to clean estimation as

$$\hat{\mathbf{x}}_{0|t}(\mathbf{y}) = \hat{\mathbf{x}}_{0|t} - \beta_t \nabla_{\hat{\mathbf{x}}_{0|t}} \log p(\mathbf{y}|\hat{\mathbf{x}}_{0|t})$$

where $\beta_t = \frac{\zeta_t}{a_t} \frac{dt}{C_1(t)}$. For a linear flow with $\sigma_t = t$,

$$\frac{\zeta_t}{a_t} = \dot{a}_t \frac{b_t}{a_t} \left(1 - \frac{b_t}{a_t}\right) = \frac{-\sigma_t}{1 - \sigma_t} \left(1 - \frac{\sigma_t}{1 - \sigma_t}\right) \quad (56)$$

and β_t is expressed as

$$\beta_t = \frac{dt\sigma_t(2\sigma_t - 1)}{(1 - \sigma_t)^2(1 - \sigma_{t+dt})} \quad (57)$$

with $dt = \sigma_{t+dt} - \sigma_t < 0$. Figure 6 illustrates the progress of $-\beta_t$ during the sampling process. As we mentioned in the main paper, it rapidly decreases to zero which leads to higher stepsize for likelihood gradient in the early stage. In our method, we imitate the theory-driven behavior of step size for likelihood gradient by introducing interpolation coefficient γ that emphasizes data consistency (i.e. likelihood gradient) in the early stage.

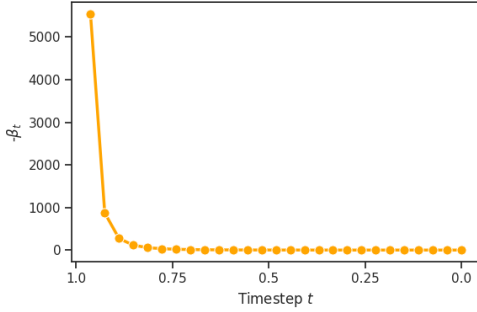


Figure 6. Evolution of $-\beta_t$ during sampling.

9.2. CFG scale and NFE

CFG scale λ is an important factor for generating high-frequency details based on provided text prompts. When using low λ , the model fails to capture fine details, producing blurry results. Using high λ enables the model to better generate fine details, but can also guide sample generation in a completely wrong direction when excessively high. Figure 7 in the Appendix shows how when using higher NFEs, the model requires more guidance for production of high-frequency details, causing higher values of λ to perform better.

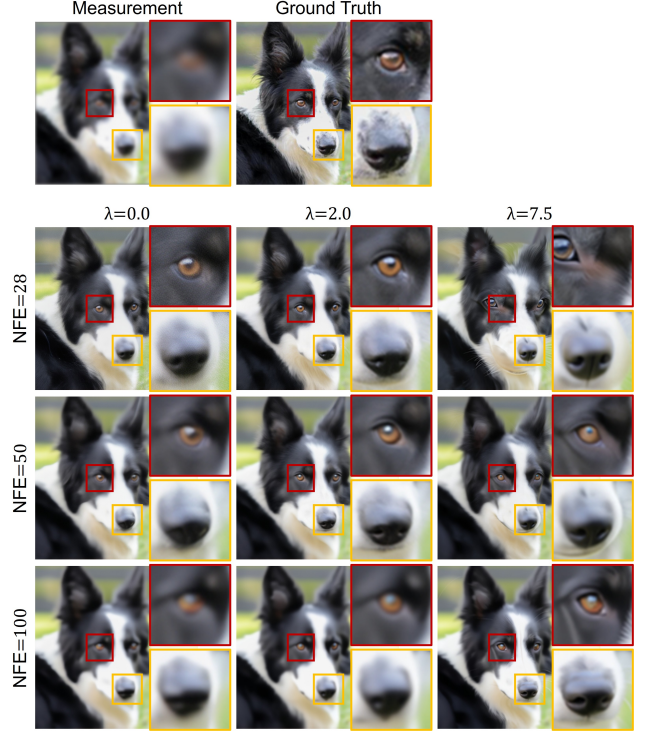


Figure 7. Relation between NFE and CFG scale λ . For varying NFEs (28, 50, 100), we study the impact of λ (0.0, 2.0, 7.5).

10. Runtime Comparison

Among baselines, PSLD takes the longest time as it requires computation of the Jacobian in terms of the transformer denoiser. In contrast, ReSample, FlowChef, and FlowDPS do not perform this computation due to approximation of the Jacobian, making them relatively efficient. Theoretically, the main computational bottleneck for these methods is backpropagation through the decoder and all other computations are relatively small, giving nearly identical runtime. Although LatentDAPS does not require backpropagation through a neural network, repeatedly solving the flow ODE and performing Langevin dynamics multiple times to achieve sufficient performance induces longer runtime compared to FlowDPS.

11. Additional Results

11.1. Higher pixel resolution

The diffusion process defined in latent space enables control over the generation process through various conditions, such as text, while also improving computational efficiency. Specifically, latent flow models like Stable Diffusion 3.0 can generate images with resolutions exceeding 1K pixels and support various aspect ratios beyond standard square formats. We leverage this advantage of latent flow models to solve inverse problems, demonstrating the practical ap-

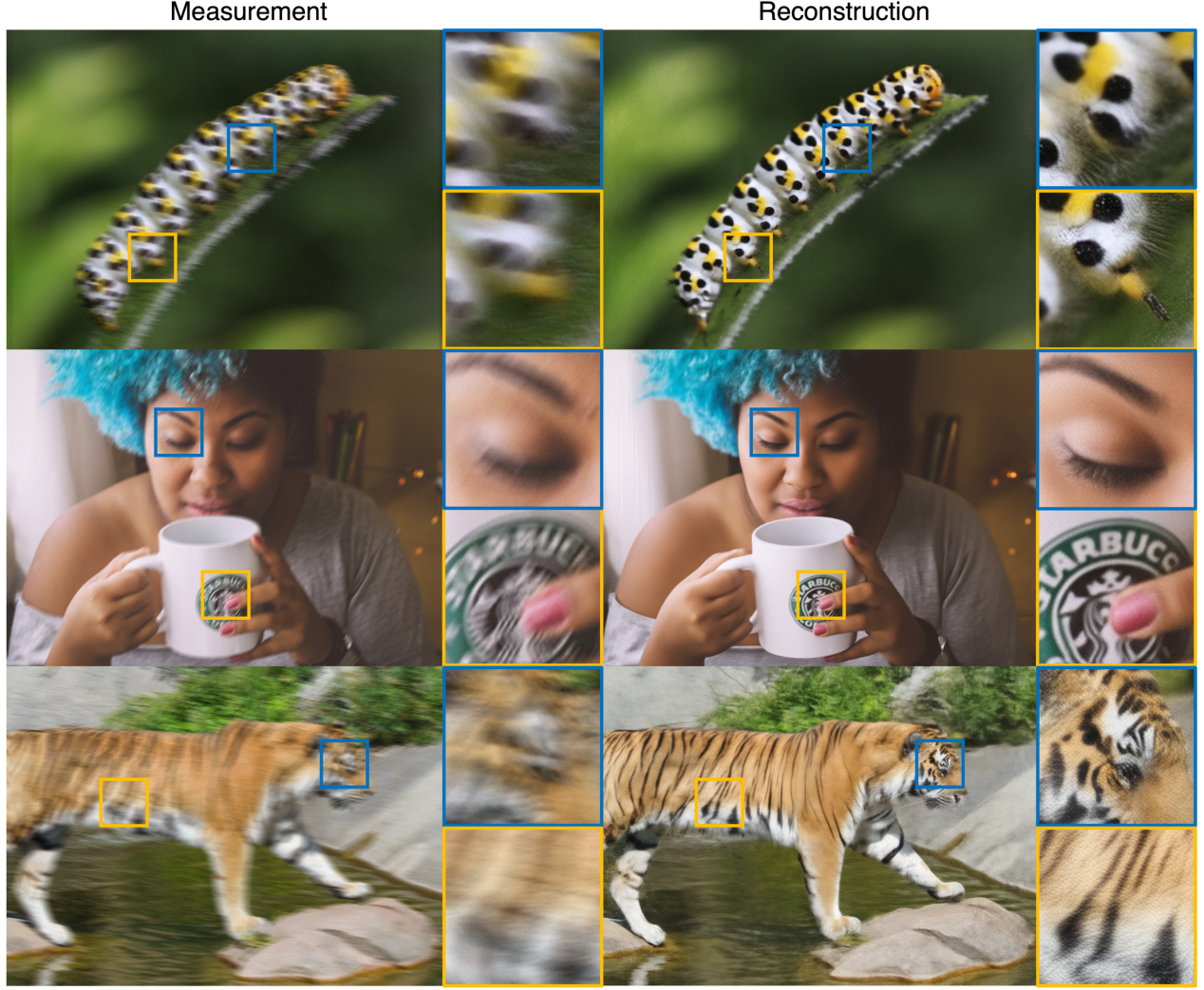


Figure 8. Motion deblur results by FlowDPS with SD3.0 for DIV2K images (1024x1408)

plicability of FlowDPS in real-world scenarios. In Figure 8, we address the motion blur problem using 1024x1408 images, obtained by cropping the central region of the DIV2K training set. For the text guidance, we use text description of the image extracted by LLaVA [13]. As demonstrated in the main experiment, FlowDPS successfully solves inverse problems defined in higher pixel resolutions without loss of generality.

11.2. Inverse problem solving with FLUX

The FlowDPS framework is designed for general affine conditional flows, enabling the construction of an inverse problem solver with various flow models. In our main experiment, we demonstrate the efficiency of FlowDPS using a pre-trained linear conditional flow model, Stable Diffusion 3. To further illustrate the generality of FlowDPS, we also incorporate another open-source linear conditional flow model, FLUX. Specifically, we set $\eta_t = 0$ and use

10 iterations for data consistency optimization. The step size is set to 7.0 for super-resolution and 12.0 for deblurring, while all other settings remain consistent with Stable Diffusion 3. Figure 9 and 10 present the reconstruction results of FlowDPS with FLUX 1.0-schnell [10] on DIV2K images. As in the main experiment, we utilize text prompts extracted from measurements using DAPE [26]. Regardless of the backbone model, FlowDPS effectively solves inverse problems.

11.3. Additional Qualitative comparisons

In this section, we further provide qualitative comparisons for the four inverse problems across the AFHQ and FFHQ datasets. Figures 11-18 show that FlowDPS consistently achieves promising reconstruction results and outperforms all the baselines.

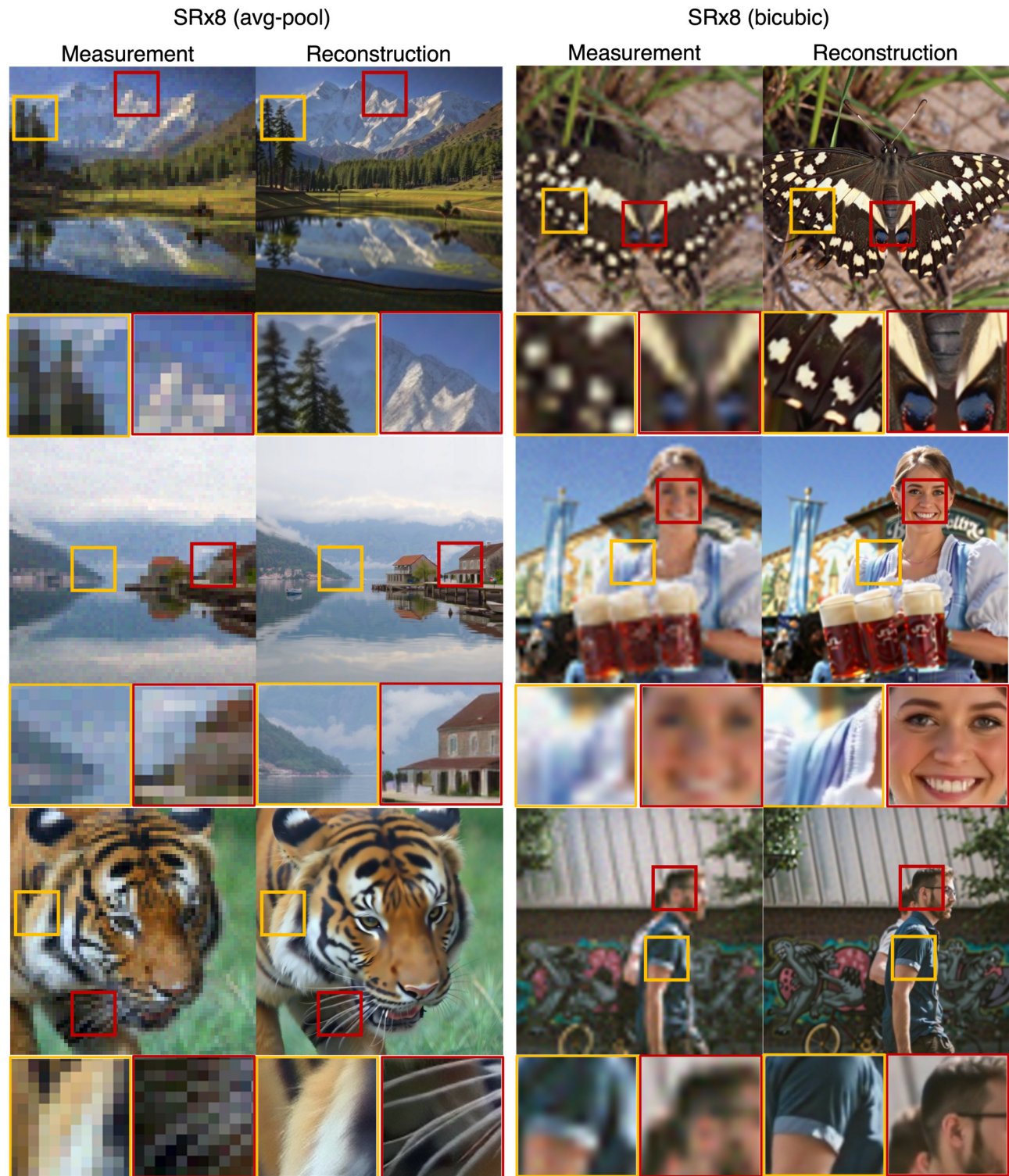


Figure 9. Super-resolution (x8) results by FlowDPS with Flux for DIV2K images (512x512)



Figure 10. Deblurring results by FlowDPS with Flux for DIV2K images (512x512)



Figure 11. Qualitative comparison for x12 super-resolution from average pooling on the AFHQ dataset.



Figure 12. Qualitative comparison for x12 super-resolution from bicubic downsampling on the AFHQ dataset.



Figure 13. Qualitative comparison for Gaussian deblurring on the AFHQ dataset.



Figure 14. Qualitative comparison for motion deblurring on the AFHQ dataset.



Figure 15. Qualitative comparison for x12 super-resolution from average pooling on the FFHQ dataset.



Figure 16. Qualitative comparison for x12 super-resolution from bicubic downsampling on the FFHQ dataset.



Figure 17. Qualitative comparison for Gaussian deblurring on the FFHQ dataset.



Figure 18. Qualitative comparison for motion deblurring on the FFHQ dataset.